

Claude Code 40 万会话研究

AI 编程从“代码工具”走向 Agentic Work Runtime

一份面向入门者的 10–15 分钟论文讲解教程

~40 万 交互式 Claude Code 会话	~23.5 万 研究涉及用户	7 个月 2025-10 → 2026-04
70% 人类负责规划决策	80% Claude 负责执行决策	+27% 估算平均任务价值增长

一句话读懂

这篇研究不是在问“AI 会不会写代码”，而是在问：真实工作中，人和 Coding Agent 如何分工？什么样的人更能让 Agent 做成事情？

核心论文：Agentic coding and persistent returns to expertise

Anthropic Economic Research • 2026-06-16

教程定位：简单入门 / 关键图解 / know-why + know-how

先知道这篇论文在回答什么

把“AI 写代码”升级成“人类如何管理一个能行动的 Agent”。



三个核心问题

- 人和 Agent 怎么分工？谁决定目标、方案和验收？谁负责读文件、写代码、跑命令？
- 什么样的人用 Agent 更有效？是“会写代码”更重要，还是“懂业务/领域”更重要？
- Agent 的用途在怎么变化？是否还停留在补代码、修 Bug，还是开始负责部署、运行、分析和文档？

读完你应该带走的 5 个概念

概念	最简单的理解
Planning decision	决定“做什么、为什么、做到什么算完成”
Execution decision	决定“改哪个文件、写什么代码、跑什么命令”
Expertise	对当前任务的领域理解，不等于职位或编程年限
Verified success	不仅模型说“完成”，还出现测试、commit 等硬证据
Action chain	一次人类指令触发 Agent 连续执行的一串动作

阅读姿势

不要把它当成“Claude Code 性能 benchmark”。它是一项真实使用行为研究：观察人和 Agent 在实际会话里的协作模式。

来源：Anthropic, “Agentic coding and persistent returns to expertise” , 2026-06-16。

40 万会话，究竟分析了什么？

样本很大，但边界也很重要。

~400,000 interactive sessions	~235,000 people	2025.10– 2026.04 观察窗口
---	---------------------------------------	---

纳入的使用方式

- Claude Code CLI
- Claude.ai 中的 Claude Code
- Claude Code desktop app

明确没有纳入

- 第三方 IDE 集成
- SDK 调用
- headless / programmatic 模式，例如 ``claude -p``

为什么这个限制重要？

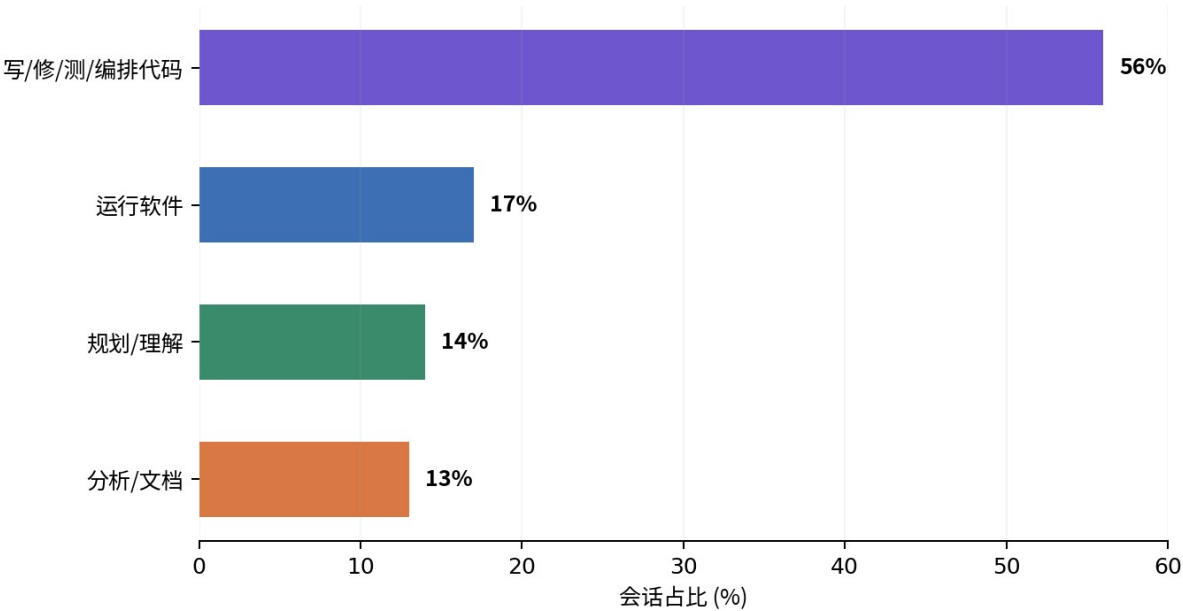
论文主要看到的是“人 ↔ Agent 的交互式协作”。而企业未来大量自动化可能发生在 CI/CD、脚本、流水线和无人值守 Agent 中；这部分论文没有覆盖。

他们不是只看 token / latency

传统 Telemetry • Token 数 • Tool call 数 • 延迟 • 会话长度	论文新增的语义层 • 工作类型 Work Mode • 人/AI 决策归属 • 用户领域 Expertise • Success / Failure • 任务估算价值
---	--

Claude Code 已经不只是“写代码”

约 44% 会话的核心目标已经不是直接写/修/测代码。



九类工作模式，可以粗分成四层

层次	典型任务	意义
代码实现	Build / Fix / Test / Orchestrate	最传统的 Coding Agent 用法
运行软件	部署、配置、运行 pipeline、监控	Agent 从“编辑器”进入“运行时”
理解与规划	读系统、理解代码、规划改动	Agent 参与 problem solving
非代码产物	数据分析、演示、文档	Coding Agent 向一般知识工作外溢

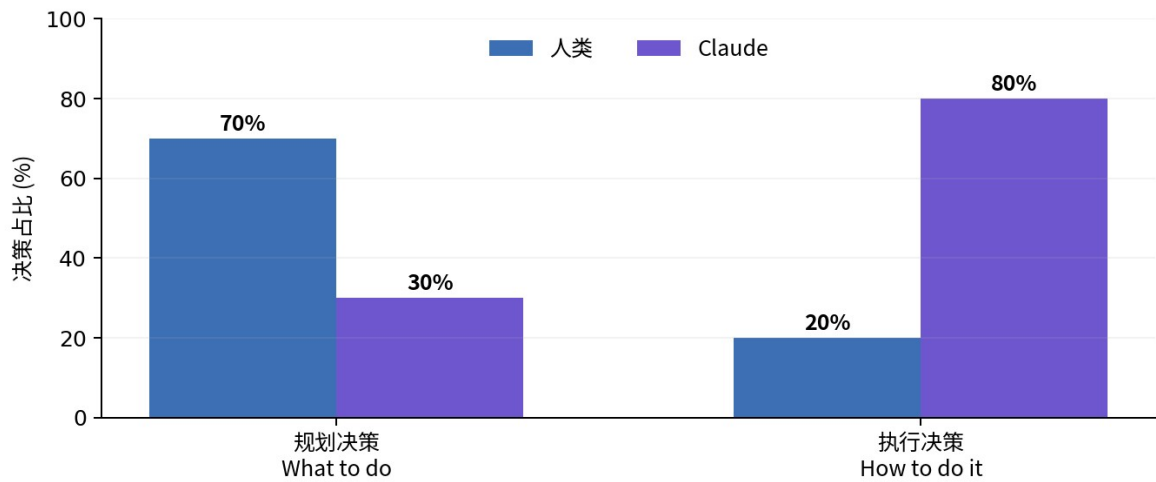
关键转变

AI Coding 的对象正在从 “code completion” 变成 “software work”：读、改、跑、测、部署、分析和交付。

来源：论文 Figure 1：56% 为写/修/测/编排代码，17% 运行软件，14% 规划/理解，13% 分析/文档。图为教程重新绘制。

最关键的分工：人决定 What，Agent 决定 How

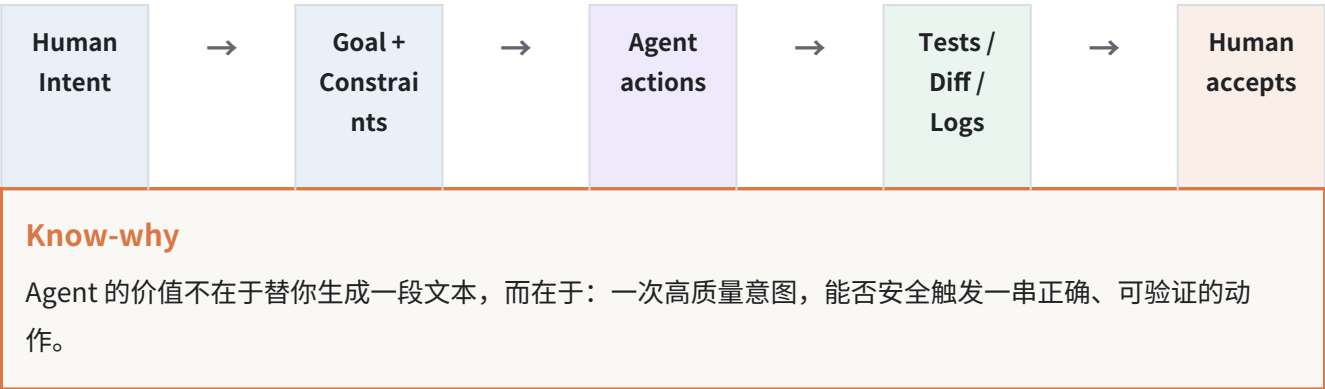
这比 “AI 写了多少行代码” 更能解释 Agentic Coding。



人类更像 “任务 Owner” - 目标是什么 - 采用什么总体方案 - 什么算完成 - 业务约束与优先级	Agent 更像 “执行 Runtime” - 读哪个文件 - 写什么实现 - 运行哪些命令 - 测试 / debug / 迭代
---	--

为什么这很重要？

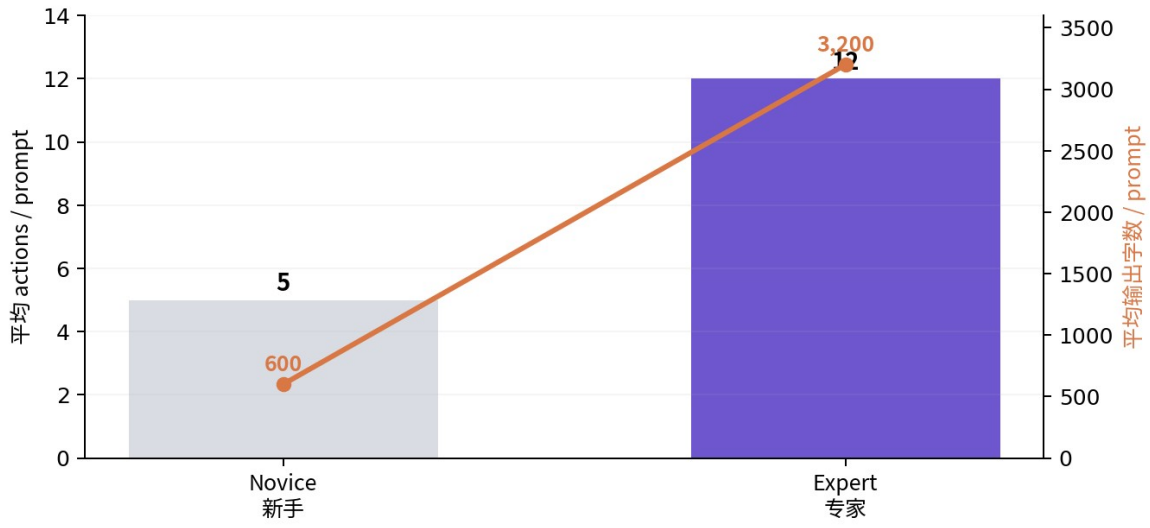
传统 IDE 时代，代码 ownership 很容易等同于 “谁敲出了这些代码”。Agent 时代更值得关注的是 “谁拥有关键决策权”。



来源：论文 Figure 2：典型会话中，人类约承担 70% planning decisions；Claude 约承担 80% execution decisions。

专家反而更能“放手让 Agent 做”

领域 expertise 会放大 Agent 的自主工作量。



论文怎么判断“专家”？

- **指令是否精确：**是否能讲清任务、边界、约束。
- **是否知道怎么验证：**会不会要求测试、检查关键 invariant、验收结果。
- **谁在纠正谁：**用户能否识别 Agent 的误解并把它拉回正确方向。

注意：Expertise 是“任务级”的

高级工程师第一次做 Rust，可能只是 Beginner；不会 Python 的会计，如果非常清楚对账规则和边界条件，仍可能是该任务的 Expert。

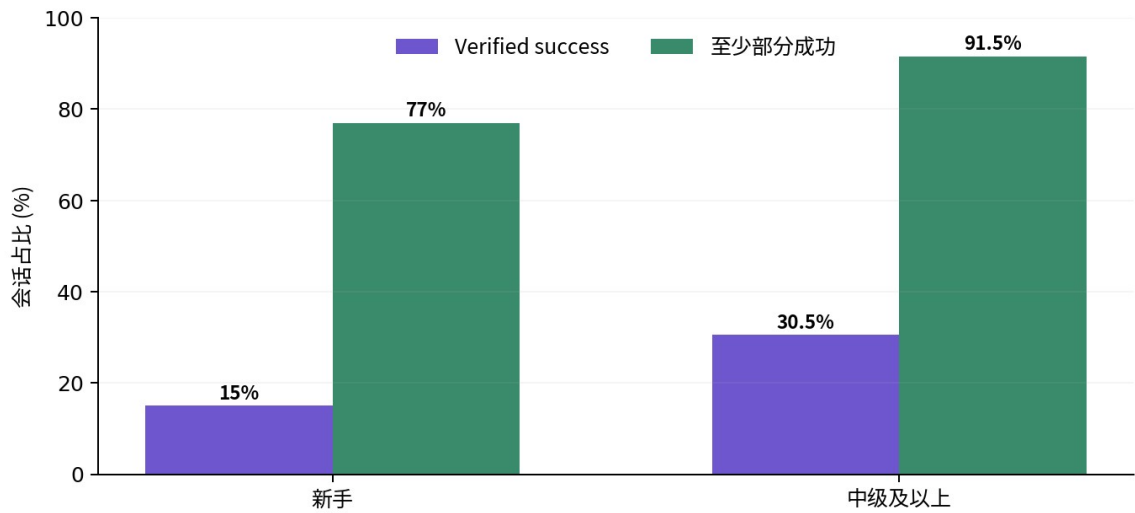
一个很实用的推导



来源：论文 Figure 3：典型新手会话约 5 actions / prompt、600 words；专家约 12 actions / prompt、3,200 words。

真正拉开差距的，是“出错之后能不能救回来”

Expertise 的价值不仅在首轮指令，更在诊断、纠偏和验收。



Verified success 是什么？

论文不是只看“Claude 说完成了”。Verified success 需要会话被判断为成功，并且至少出现一个硬验证信号，例如匹配工作的 git commit / PR、测试通过，或用户明确确认。

情景	新手	更高 expertise
Verified success	约 15%	Intermediate 及以上约 28–33%
至少部分成功	约 77%	约 91–92%
遇到 trouble 后仍 verified success	约 4%	Expert 约 15%
遇到 trouble 后放弃	约 19%	其他 expertise 约 5–7%

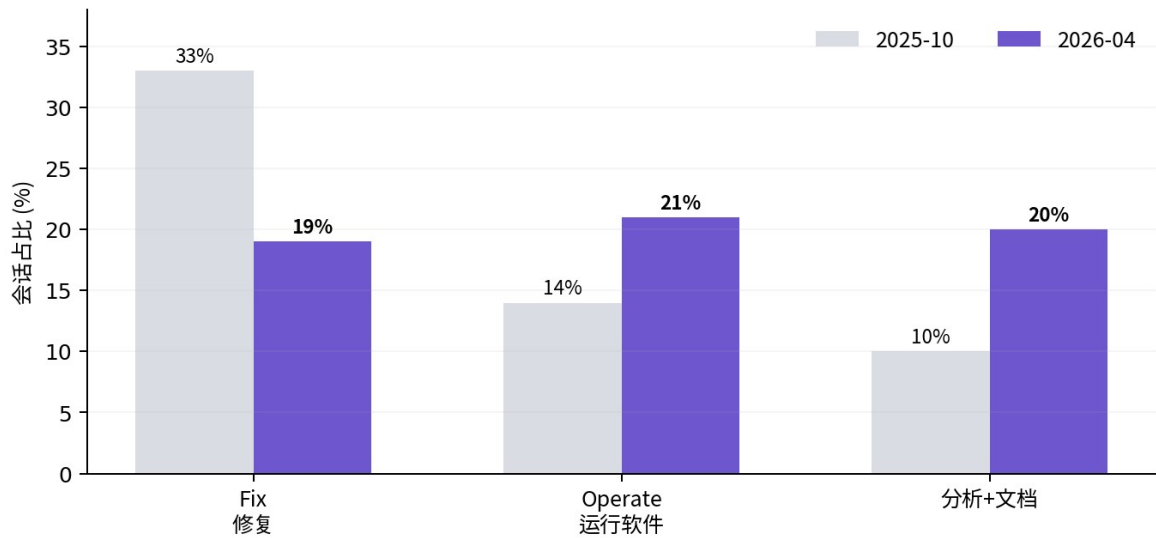
核心启发

未来人的高价值技能会越来越像：发现失败 → 解释失败 → 重设约束 → 再验证，而不仅是亲手完成 implementation。

来源：论文 Figure 5 与正文：新手 vs intermediate/expert 的成功率、trouble recovery 与 abandonment。图中 30.5% / 91.5% 为论文给出的区间中点，仅用于入门视觉对比。

从 Fix Bug，走向 Operate / Analyze / Deliver

七个月里，Claude Code 的工作结构已经出现明显迁移。



论文观察到的方向

- **Fix:** 33% → 19%，占比接近减半。
- **Operate:** 14% → 21%，部署、运行和系统操作增加。
- **分析 + 文档:** 约 10% → 20%，非代码产物明显增加。
- **任务估算价值:** 平均约 +27%；Build / Operate / Fix 分别约 +43% / +34% / +32%。

不要把 +27% 当成真实美元 ROI

论文用类似 freelancer job posting 的方法估算任务相对价值。作者明确提醒：更适合看“相对变化与排序”，不适合把美元绝对值当真。

一个更大的产品演化图



来源：论文 Figure 4 与正文。任务价值估计为粗略相对指标。

“会不会写代码”正在变得没那么决定性

在会产生代码的 session 中，不同职业的成功率差距并没有想象中大。



该怎么理解？

错误理解 “经理比程序员更会写代码” “编程能力已经没用了” “任何人都能稳定产出生产代码”	更合理的理解 - Coding occupation 不再是唯一门槛 - 领域知识能通过 Agent 转化为技术产出 - 仍然需要验证、边界与责任
---	--

论文还发现：代码产出会话中，十大职业组的 verified success 都在软件/数学职业的 7 个百分点以内。管理职业略高，但作者特别提醒这可能部分来自测量偏差。

真正可能被商品化的是 implementation

当 Agent 接管越来越多“怎么实现”，人的稀缺价值会更集中到：Problem framing、Domain judgment、Architecture、Acceptance criteria 和 Verification。

来源：论文 Figure 6 与正文：software-related vs non-software occupations 的成功率比较；作者同时提示 manager 高成功率可能受验证方式影响。

为什么会有人说它已是“企业基础设施级收入”？

行为研究 + 商业数据放在一起，才构成这个判断。



务必分清证据来源

上面三个数字来自 Anthropic 2026-02-12 的融资公告，不来自 40 万会话论文。论文证明“怎么用、谁更成功、用途如何迁移”；商业公告证明“这类产品已经形成大规模收入与企业采用”。

把两组证据拼起来



为什么 Coding Agent 最先商业化？

软件工程天然拥有强反馈闭环：文件、编译器、测试、日志、git diff、CI。Agent 每行动一次，都能快速得到可观察结果，再继续修正。这比很多“正确性难自动验证”的知识工作更适合率先形成可靠 Agent。

来源：商业背景：Anthropic Series G announcement, 2026-02-12。论文与商业公告是两类不同证据。

如果你在做 Agent Runtime，这篇论文能沉淀什么？

把行为研究转换成可执行的产品与架构原则。

论文发现	可沉淀的 Agent 设计原则
人类掌握多数 planning	保留清晰的 Goal / Constraints / Acceptance Criteria 接口
Agent 掌握多数 execution	Runtime 要擅长自主工具调用，而不是频繁反问
专家触发更长 action chain	提高 specification 质量，比单纯“写更长 prompt”更重要
专家更会 recovery	为失败设计 escalation、诊断、回滚、重试与人类接管
Verified success 依赖硬信号	用 test / lint / diff / eval / telemetry 验证，不要只信“Done”
Operate / 分析 / 文档上升	工具集要从 editor 扩到 shell、部署、数据、文档与业务系统

推荐的 Runtime 心智模型



最小实践清单

- 任务先写清 Goal / Constraints / Done Definition；高风险动作保留权限边界。
- 优先把验证工具化：test、lint、type-check、diff、eval。
- Telemetry 不只记 token：还要记录 task / autonomy / outcome / recovery。

这篇论文“说明了什么”，又“没有说明什么”？

入门阅读最容易踩坑的地方。

它比较有力地说明	它没有直接证明
• 真实交互里存在清晰的 What/How 分工 • 领域 expertise 与成功率、委托深度相关 • Claude Code 的用途正向端到端工作扩张 • 非软件职业也能通过 Agent 完成代码任务	• 开发者生产率提升了多少倍 • 企业因此减少了多少人力 • 每个 session 创造了多少真实美元价值 • headless / CI/CD Agent 的完整使用结构

方法论局限

- 这是 observational study，不是随机对照实验；相关性不等于因果。
- Success / Expertise / Occupation 等大量变量由模型读取 transcript 后分类。
- 研究看不到代码之后是否真正上线、被长期使用或创造了经济价值。
- 非交互式 / programmatic 使用被排除，而这可能正是企业 Agent 的重要部分。

最终一句话

Coding Agent 没有简单地“替代程序员”；它正在吸收更多 implementation，并把人的价值推向 problem ownership、领域理解、specification、verification 和 judgment。

30 秒复习

- 人类：约 70% planning；Claude：约 80% execution。
- 专家：每次 prompt 触发更多 actions，而且更会从失败中恢复。
- 用途：从 Fix 扩展到 Operate、Analyze、Document。
- 职业：coding background 的门槛在下降，domain expertise 的价值仍然明显。
- 工程：把一次人类意图转成更长、更安全、更可验证的 action trajectory。

来源

本教程以 Anthropic 官方研究与官方商业公告为主。

#	资料	用途
[1]	Anthropic Economic Research. Agentic coding and persistent returns to expertise. 2026-06-16.	论文主体：样本、work mode、决策分工、expertise、success、趋势与局限
[2]	Anthropic. Series G funding announcement. 2026-02-12.	商业背景：Claude Code run-rate revenue >\$2.5B, enterprise use > half of revenue

官方链接（可复制）

<https://www.anthropic.com/research/claude-code-expertise>
<https://www.anthropic.com/news/anthropic-raises-30-billion-series-g-funding-380-billion-post-money-valuation>

建议下一步怎么读

先看论文正文 Figure 1–6，理解“工作类型 → 人/AI 分工 → expertise → success”的主线；再读 Appendix，重点看 classifier、任务价值估算和 validation。

— END —